23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No	C4
Project Title	Apex Bank Management System

Team Members

Roll No	Student Name
AM.SC.U4CSE25225	Kaki Venkata Pavithra
AM.SC.U4CSE25250	Singuru Chathurya
AM.SC.U4CSE25257	Harshini Vedulla
AM.SC.U4CSE25268	Gara Sriha

Project Overview

Apex Bank Management System is a relational-database-backed retail banking platform that connects Customers and Administrators on a single system, replacing manual, ledger-based record keeping with automated account management, fund transfers, card issuance, nominee registration and loan processing. The system is implemented on PostgreSQL and its web application exposes two portals, Customer and Admin. This document focuses on the database layer that underpins that platform: the ER model, the relational schema derived from it, and the normalization analysis that justifies the schema's design.

Contents of this Document

Section	Title	Covers
3.1	Detailed Description of the Project	Scenario, entities/attributes, users and business rules
3.2	ER Diagram	ER model with cardinality and participation constraints
3.3	Relational Schema	10 relations mapped from the ER diagram, with keys and constraints
3.4	Normalization & FD Analysis	Functional dependencies, candidate-key closure, 1NF to BCNF decomposition

Reference Material

This design document should be read alongside the Project Abstract (problem statement and objectives) and the UI Design Document (functional/non-functional requirements and screen designs) submitted separately by Group C4.

Live demo: https://dbms-project-m3ui.vercel.app/

Database Design Highlights

- Role-based access (Customer, Employee, Admin) modelled with a disjoint, total EER specialisation on a single USER entity. The deployed application exposes the Customer and Admin portals.
- Historical ledger integrity preserved by treating TRANSACTION as an append-only weak entity: rows are never updated or deleted, and BalanceAfter snapshots the running balance at the time of each entry.
- Every relation in the final schema is verified to be in BCNF (Section 3.4), including the alternate candidate keys on USER, BRANCH, ACCOUNT and CARD.
- Implemented in PostgreSQL with SERIAL keys, TIMESTAMP columns and CHECK-constrained VARCHAR columns in place of ENUM types.

3.1 Detailed Description of the Project

The Apex Bank Management System is a database-driven application that automates core retail-banking operations — account opening, deposits and withdrawals, fund transfers, card issuance, nominee registration and loan processing — replacing manual, ledger-based record keeping with a single relational platform. Two categories of users operate the deployed system: Customers and Administrators (who audit and manage the system). Employees (branch staff) exist as a role in the database and are the optional processors of loans, but they have no separate portal. The database stores and relates data about user identities, branches, accounts, transactions, nominees, cards and loans, while enforcing integrity constraints (primary/foreign keys, uniqueness, domain and check constraints) so that financial data stays accurate, consistent and non-redundant.

Entities and their attributes

- **User** — every person who can log in (UserID, Name, Email, Phone, Password, Role, CreatedAt). Role-specific data is captured through specialisation into Customer (Address, City, DateOfBirth, KYCStatus), Employee (BranchID, Designation, Salary) and Admin (AccessLevel).
- **Branch** — a physical bank branch (BranchID, BranchName, IFSCCode, Address, City, ContactNumber, IsActive).
- **Account** — a bank account opened by exactly one customer at exactly one branch (AccountID, AccountNumber, CustomerID, BranchID, AccountType, Balance, OpenedDate, Status).
- **Transaction** — a single ledger entry against one account (AccountID, TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description).
- **Nominee** — a beneficiary registered against one account (AccountID, NomineeSeqNo, NomineeName, Relationship, SharePercentage).
- **Card** — the card issued for one account (CardID, AccountID, CardNumber, CardType, ExpiryDate, CVV, Status).
- **Loan** — a loan applied for by one customer and optionally assigned to one employee (LoanID, CustomerID, EmployeeID, LoanType, PrincipalAmount, InterestRate, TenureMonths, LoanStatus, DisbursedDate).

Business rules

1. Every USER row belongs to exactly one of three roles (disjoint, total specialisation); role-specific attributes live only in the matching subclass table, which shares UserID as primary key and foreign key. Deleting a USER cascades to its subclass row.
2. An Employee is affiliated with exactly one Branch, but a Branch may employ many Employees.
3. A Customer may own zero or more Accounts; each Account is owned by exactly one Customer and is opened at exactly one Branch (joint accounts are out of scope for this design).
4. An Account accumulates zero or more Transactions; a Transaction cannot exist without referencing a valid Account. Once written, a Transaction row is never updated or deleted — the ledger is append-only, and the current Balance on Account is a maintained summary of its Transactions.
5. An Account may register zero or more Nominees; a Nominee cannot exist without a valid Account, and the SharePercentage values of all Nominees on one Account must sum to 100 (enforced by application logic).
6. An Account has at most one Card (CARD.AccountID is UNIQUE and NOT NULL) and a Card belongs to exactly one Account. A card cannot exist without an account, and deleting an account removes its card.
7. A Customer may apply for zero or more Loans; each Loan is applied for by exactly one Customer. A Loan may be assigned to one Employee for processing; the assignment is optional (EmployeeID is nullable), so a newly applied loan has no processor yet, and one Employee may process many Loans over time.

8. LoanStatus follows the sequence Applied → UnderReview → Approved → Disbursed → Closed (or Rejected / Defaulted at intermediate points); the database enforces the allowed values and the default of Applied, while the transition order is kept consistent through application logic.
9. Administrators do not take part in the transactional relationships above; they hold read access for auditing and reporting (transaction, loan and user reports) and can freeze or deactivate any User account or Account (Status = Frozen / Closed).

3.2 ER Diagram

Chen notation is used: rectangles are entities (double-bordered = weak entity), diamonds are relationships (double diamond = identifying relationship) and the circle marked d denotes the disjoint specialisation. Only entity names are shown in the diagram; the attributes of each entity are listed in Section 3.3. USER is modelled with disjoint, total specialisation into CUSTOMER, EMPLOYEE and ADMIN based on the Role attribute; ADMIN is not linked by any transactional relationship since it only performs cross-cutting auditing/management. The labels 1 and M give the cardinality of each relationship.

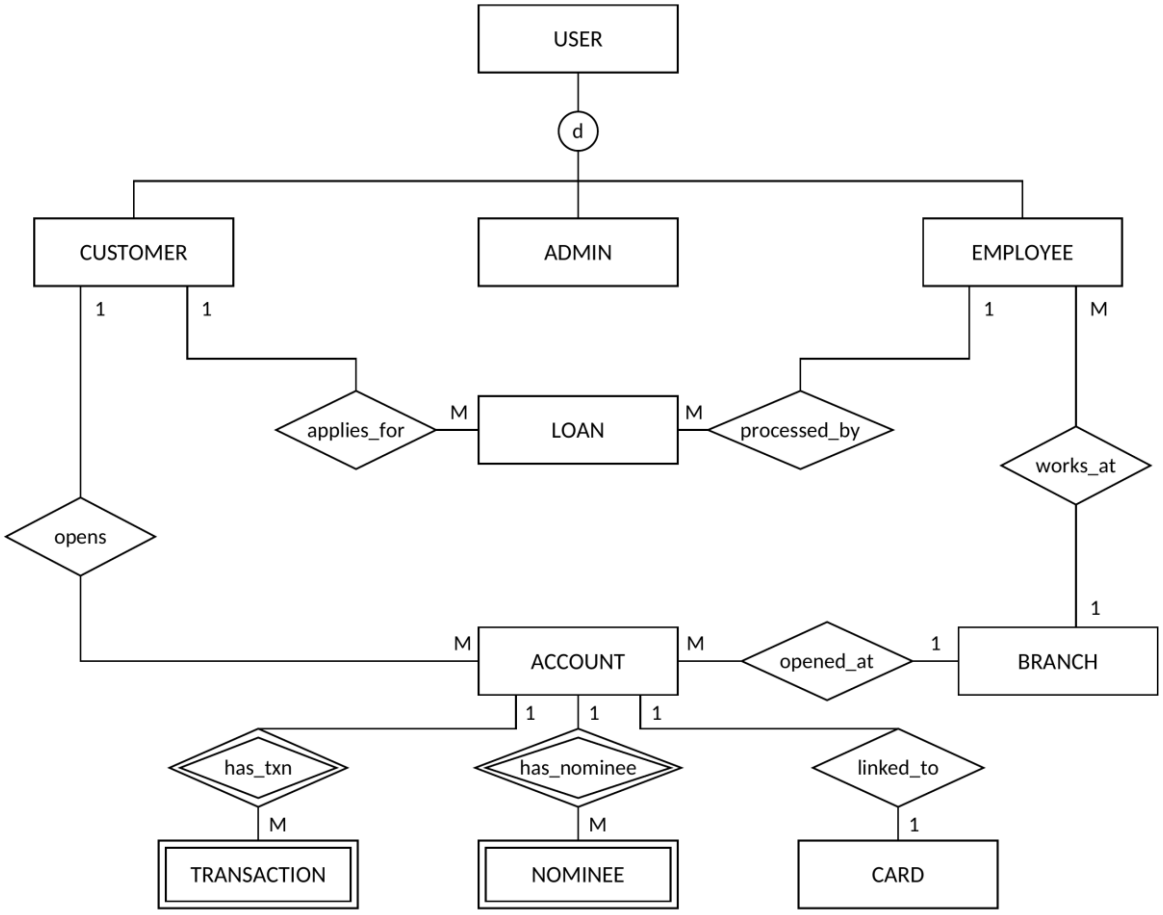


Fig. 3.1 — ER Diagram (Chen notation) of the Apex Bank Management System database

Entity classification

Entity	Type	Identifier
USER	Strong (superclass)	UserID
CUSTOMER, EMPLOYEE, ADMIN	Subclasses of USER	UserID (inherited)
BRANCH	Strong	BranchID

Entity	Type	Identifier
ACCOUNT	Strong	AccountID (AccountNumber is an alternate key)
CARD	Strong (dependent on ACCOUNT through a 1:1 relationship)	CardID (AccountID and CardNumber are alternate keys)
LOAN	Strong	LoanID
TRANSACTION	Weak, owner ACCOUNT	AccountID + TransactionSeqNo
NOMINEE	Weak, owner ACCOUNT	AccountID + NomineeSeqNo

Cardinality and participation summary

Relationship	Entities	Cardinality	Participation
is_a (Role)	USER → CUSTOMER / EMPLOYEE / ADMIN	1 : 0..1 each	Disjoint, total
works_at	EMPLOYEE – BRANCH	M : 1	Total (Employee) / Partial (Branch)
opens	CUSTOMER – ACCOUNT	1 : M	Total (Account) / Partial (Customer)
opened_at	ACCOUNT – BRANCH	M : 1	Total (Account) / Partial (Branch)
has_txn (identifying)	ACCOUNT – TRANSACTION	1 : M	Total (Transaction) / Partial (Account)
has_nominee (identifying)	ACCOUNT – NOMINEE	1 : M	Total (Nominee) / Partial (Account)
linked_to	ACCOUNT – CARD	1 : 1	Total (Card) / Partial (Account)
applies_for	CUSTOMER – LOAN	1 : M	Total (Loan) / Partial (Customer)
processed_by	EMPLOYEE – LOAN	1 : M	Partial (both sides; Loan.EmployeeID is nullable)

3.3 Relational Schema

The ER diagram is mapped to relations using standard rules: each strong entity becomes a relation keyed on its identifier; the total, disjoint specialisation of USER is implemented with one relation per subclass, each sharing UserID as both primary key and foreign key back to USER; each weak entity (TRANSACTION, NOMINEE) becomes a relation whose primary key is the composite of its own discriminator and the owner entity's key; 1:M relationships place the foreign key on the 'many' side (EMPLOYEE.BranchID, ACCOUNT.CustomerID/BranchID, LOAN.CustomerID/EmployeeID); and the 1:1 relationship (Account–Card) places a unique foreign key on the dependent relation. All SERIAL columns are auto-incrementing INT primary keys.

USER (written "User" in SQL because USER is a reserved word)

Attribute	Type	Key	Constraint / Default
UserID	SERIAL	PK	
Name	VARCHAR(80)		NOT NULL
Email	VARCHAR(100)		UNIQUE, NOT NULL
Phone	VARCHAR(15)		NOT NULL
Password	VARCHAR(255)		NOT NULL (hashed)
Role	VARCHAR(20)		NOT NULL, CHECK IN (Customer, Employee, Admin)
CreatedAt	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP

CUSTOMER (subclass of USER)

Attribute	Type	Key	Constraint / Default
UserID	INT	PK, FK	References "User"(UserID) ON DELETE CASCADE

Attribute	Type	Key	Constraint / Default
Address	VARCHAR(150)		
City	VARCHAR(50)		
DateOfBirth	DATE		
KYCStatus	VARCHAR(20)		DEFAULT 'Pending', CHECK IN (Pending, Verified, Rejected)

EMPLOYEE (subclass of USER)

Attribute	Type	Key	Constraint / Default
UserID	INT	PK, FK	References "User"(UserID) ON DELETE CASCADE
BranchID	INT	FK	NOT NULL, References Branch(BranchID)
Designation	VARCHAR(40)		
Salary	DECIMAL(10,2)		CHECK (Salary > 0)

ADMIN (subclass of USER)

Attribute	Type	Key	Constraint / Default
UserID	INT	PK, FK	References "User"(UserID) ON DELETE CASCADE
AccessLevel	VARCHAR(20)		

BRANCH

Attribute	Type	Key	Constraint / Default
BranchID	SERIAL	PK	
BranchName	VARCHAR(100)		NOT NULL
IFSCCode	VARCHAR(11)		UNIQUE, NOT NULL
Address	VARCHAR(150)		NOT NULL
City	VARCHAR(50)		NOT NULL
ContactNumber	VARCHAR(15)		
IsActive	BOOLEAN		DEFAULT TRUE

ACCOUNT

Attribute	Type	Key	Constraint / Default
AccountID	SERIAL	PK	
AccountNumber	VARCHAR(18)		UNIQUE, NOT NULL
CustomerID	INT	FK	NOT NULL, References CUSTOMER(UserID)
BranchID	INT	FK	NOT NULL, References BRANCH(BranchID)
AccountType	VARCHAR(20)		NOT NULL, CHECK IN (Savings, Current, Fixed Deposit)
Balance	DECIMAL(12,2)		DEFAULT 0, CHECK >= 0
OpenedDate	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP
Status	VARCHAR(20)		DEFAULT 'Active', CHECK IN (Active, Frozen, Closed)

TRANSACTION (weak entity — owner: ACCOUNT)

Attribute	Type	Key	Constraint / Default
AccountID	INT	PK, FK	References ACCOUNT(AccountID)
TransactionSeqNo	INT	PK (discriminator)	Sequential per account
TransactionType	VARCHAR(20)		NOT NULL, CHECK IN (Deposit, Withdrawal, TransferIn, TransferOut)
Amount	DECIMAL(12,2)		CHECK > 0
TransactionDate	TIMESTAMP		DEFAULT CURRENT_TIMESTAMP
BalanceAfter	DECIMAL(12,2)		NOT NULL, snapshot of balance after this entry
Description	VARCHAR(255)		

NOMINEE (weak entity — owner: ACCOUNT)

Attribute	Type	Key	Constraint / Default
AccountID	INT	PK, FK	References ACCOUNT(AccountID)
NomineeSeqNo	INT	PK (discriminator)	
NomineeName	VARCHAR(80)		NOT NULL
Relationship	VARCHAR(30)		
SharePercentage	DECIMAL(5,2)		CHECK 0–100

CARD

Attribute	Type	Key	Constraint / Default
CardID	SERIAL	PK	
AccountID	INT	FK	UNIQUE, NOT NULL, References ACCOUNT(AccountID) ON DELETE CASCADE
CardNumber	VARCHAR(16)		UNIQUE, NOT NULL
CardType	VARCHAR(20)		NOT NULL, CHECK IN (Debit, Credit)
ExpiryDate	DATE		NOT NULL
CVV	VARCHAR(255)		NOT NULL (hashed)
Status	VARCHAR(20)		DEFAULT 'Active', CHECK IN (Active, Blocked, Expired)

LOAN

Attribute	Type	Key	Constraint / Default
LoanID	SERIAL	PK	
CustomerID	INT	FK	NOT NULL, References CUSTOMER(UserID)
EmployeeID	INT	FK	NULLABLE, References EMPLOYEE(UserID)
LoanType	VARCHAR(30)		NOT NULL, CHECK IN (Personal, Home, Auto, Education)
PrincipalAmount	DECIMAL(12,2)		CHECK > 0
InterestRate	DECIMAL(4,2)		CHECK > 0
TenureMonths	INT		CHECK > 0
LoanStatus	VARCHAR(20)		DEFAULT 'Applied', CHECK IN (Applied, UnderReview, Approved, Rejected, Disbursed, Closed, Defaulted)

Attribute	Type	Key	Constraint / Default
DisbursedDate	TIMESTAMP		NULL until Disbursed

3.4 Normalization and Functional Dependency Analysis

To demonstrate the normalization process, consider the Account Statement Report that a naive, unnormalised design might use for generating account statements (the same redundancy the database design in Section 3.3 is built to avoid). We begin with the unnormalised form (UNF), in which one row describes an account and contains a repeating group of its transactions:

ACCOUNT_STATEMENT (AccountID, AccountNumber, AccountType, CustomerID, CustomerName, CustomerPhone, BranchID, BranchName, IFSCCode, BranchCity, **{ TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description }**)

The braces mark a repeating group: one account holds many transactions inside a single row.

1NF — First Normal Form

A relation is in 1NF when every attribute is atomic and there are no repeating groups. We remove the repeating group by giving each transaction its own row and repeating the account facts on every row. The key becomes composite:

STATEMENT_REPORT (AccountID, TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description, AccountNumber, AccountType, CustomerID, CustomerName, CustomerPhone, BranchID, BranchName, IFSCCode, BranchCity) — **16 attributes**.

Functional dependencies identified

FD	Dependency	Source in the schema
FD1	{AccountID, TransactionSeqNo} → TransactionType, Amount, TransactionDate, BalanceAfter, Description	TRANSACTION primary key
FD2	AccountID → AccountNumber, AccountType, CustomerID, BranchID	ACCOUNT primary key
FD3	CustomerID → CustomerName, CustomerPhone	USER primary key (Name, Phone)
FD4	BranchID → BranchName, IFSCCode, BranchCity	BRANCH primary key
FD5	AccountNumber → AccountID	ACCOUNT.AccountNumber is UNIQUE
FD6	IFSCCode → BranchID	BRANCH.IFSCCode is UNIQUE

Candidate key justification using attribute closure

Step 1 — Closure of {AccountID, TransactionSeqNo}:

- {AccountID, TransactionSeqNo}$^{+}$ = { AccountID, TransactionSeqNo }
- apply FD1 → + TransactionType, Amount, TransactionDate, BalanceAfter, Description
- apply FD2 (AccountID present) → + AccountNumber, AccountType, CustomerID, BranchID
- apply FD3 (CustomerID present) → + CustomerName, CustomerPhone
- apply FD4 (BranchID present) → + BranchName, IFSCCode, BranchCity

The closure contains **all 16 attributes** of STATEMENT_REPORT, so {AccountID, TransactionSeqNo} is a super key. Neither AccountID alone (it never reaches the FD1 attributes) nor TransactionSeqNo alone has this property, so the key is minimal: **candidate key 1 = {AccountID, TransactionSeqNo}**. Because of FD5, AccountNumber determines AccountID, so **candidate key 2 = {AccountNumber, TransactionSeqNo}** is also minimal. Candidate key 1 is used as the primary key.

2NF — Second Normal Form

A relation is in 2NF when it is in 1NF and no non-prime attribute depends on only part of a candidate key. The key is composite, so partial dependencies are possible, and here they exist: AccountID → AccountNumber, AccountType, CustomerID, BranchID (FD2) depends on only part of the key, and CustomerName, CustomerPhone, BranchName, IFSCCode and BranchCity also depend on AccountID alone through CustomerID and BranchID. STATEMENT_REPORT therefore **violates 2NF**: the account facts are repeated on every transaction row, causing update anomalies (changing a customer's phone means editing every transaction) and deletion anomalies.

Decomposition to remove the partial dependencies:

- TRANSACTION_2NF (AccountID, TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description)
- ACCOUNT_2NF (AccountID, AccountNumber, AccountType, CustomerID, CustomerName, CustomerPhone, BranchID, BranchName, IFSCCode, BranchCity)

The decomposition is lossless: the shared attribute AccountID is the key of ACCOUNT_2NF.

3NF — Third Normal Form

A relation is in 3NF when it is in 2NF and no non-prime attribute depends transitively on the key. TRANSACTION_2NF is already in 3NF. ACCOUNT_2NF still contains two transitive dependencies: AccountID → CustomerID → CustomerName, CustomerPhone, and AccountID → BranchID → BranchName, IFSCCode, BranchCity. CustomerID and BranchID are non-key attributes that determine other non-key attributes, so ACCOUNT_2NF **violates 3NF**.

Decomposition to remove the transitive dependencies:

- ACCOUNT_3NF (AccountID, AccountNumber, AccountType, CustomerID, BranchID)
- CUSTOMER_3NF (CustomerID, CustomerName, CustomerPhone)
- BRANCH_3NF (BranchID, BranchName, IFSCCode, BranchCity)
- TRANSACTION_3NF = TRANSACTION_2NF (AccountID, TransactionSeqNo, TransactionType, Amount, TransactionDate, BalanceAfter, Description)

All four relations are in 3NF, and the decomposition is lossless and dependency-preserving (FD1 lives in TRANSACTION_3NF, FD2 and FD5 in ACCOUNT_3NF, FD3 in CUSTOMER_3NF, FD4 and FD6 in BRANCH_3NF).

BCNF — Boyce–Codd Normal Form

A relation is in BCNF when the determinant of every non-trivial FD is a super key. Checking each relation from the 3NF decomposition:

Relation	FD(s) present	Determinant is a key?
TRANSACTION_3NF	{AccountID, TransactionSeqNo} → all other attributes	Yes — BCNF
ACCOUNT_3NF	AccountID → rest; AccountNumber → AccountID and rest	Yes — both are candidate keys; BCNF
CUSTOMER_3NF	CustomerID → CustomerName, CustomerPhone	Yes — BCNF
BRANCH_3NF	BranchID → rest; IFSCCode → BranchID and rest	Yes — both are candidate keys; BCNF

Every determinant in every relation is a candidate key, so no further decomposition is required on FD grounds — the schema is in BCNF.

Mapping back to the final schema (Section 3.3)

This textbook decomposition motivates the final design: TRANSACTION_3NF → TRANSACTION (same key {AccountID, TransactionSeqNo}, modelled as a weak entity of ACCOUNT); ACCOUNT_3NF → ACCOUNT (extended with Balance, OpenedDate and Status); CUSTOMER_3NF → USER + CUSTOMER (CustomerName and CustomerPhone are stored once in USER as Name and Phone, and the customer-specific attributes sit in CUSTOMER); BRANCH_3NF → BRANCH (BranchCity is BRANCH.City, extended with Address, ContactNumber and IsActive).

EMPLOYEE, ADMIN, CARD, NOMINEE and LOAN were not part of the original flat report, so they do not appear in the FD analysis above; they were added directly at the ER stage as entities in their own right, each keyed on its own identifier (or composite key, for the weak entity). Keeping them as separate relations is lossless and avoids repeating card, nominee or loan data across every transaction row, which is the same normalisation principle applied one level up.

BCNF verification of all 10 relations in the final schema

Relation	Candidate keys	Non-trivial FDs	BCNF
USER	UserID, Email	UserID → all; Email → all	Yes
CUSTOMER	UserID	UserID → Address, City, DateOfBirth, KYCStatus	Yes
EMPLOYEE	UserID	UserID → BranchID, Designation, Salary	Yes
ADMIN	UserID	UserID → AccessLevel	Yes
BRANCH	BranchID, IFSCCode	each key → all	Yes
ACCOUNT	AccountID, AccountNumber	each key → all	Yes
CARD	CardID, AccountID, CardNumber	each key → all	Yes
LOAN	LoanID	LoanID → all	Yes
TRANSACTION	{AccountID, TransactionSeqNo}	key → all	Yes
NOMINEE	{AccountID, NomineeSeqNo}	key → all	Yes

Deliberate redundancy and integrity notes

- **Derived data.** ACCOUNT.Balance can be computed from TRANSACTION, and TRANSACTION.BalanceAfter is a snapshot of the running balance. Both are deliberate denormalisations for fast reads and statement printing; they are not normal-form violations because neither is determined by a non-key attribute. The application updates Balance and inserts the ledger row in one database transaction.
- **Role consistency.** USER.Role and the existence of a row in CUSTOMER, EMPLOYEE or ADMIN must agree. The CHECK constraint restricts Role to three values; agreement with the subclass row is maintained by application logic.
- **Rules held by the application.** The append-only ledger, nominee shares totalling 100, and the LoanStatus sequence are handled in application logic.
- **Hashed secrets.** Password and CVV are stored as hashes (VARCHAR(255)). Storing CVV, even hashed, is discouraged by PCI-DSS and is acceptable here only because this is an academic project.